

Shrub Hub

CHANGING OUTPUTS GUIDE



Introduction

The Shrub Hub comes with a total of twenty-five different outputs, split into five modes of five outputs each. These outputs were chosen based on existing digital interfaces and input from clinicians, but they may not be the ideal outputs for your use case. If that is the case, you can change the outputs that the Shrub Hub has to better suit your needs.

The different outputs of the various modes can be modified by a skilled Maker or Secondary User.

Feedback

If you do change the outputs for your Shrub Hub, please get in touch with us to tell us what you changed them to so we can better understand the needs of the Shrub Hub users and change the default outputs to better represent the needs of the users.

You can get in touch with us in any of the following ways

- Email us at info@makersmakingchange.com
- File a GitHub Issue at the following link: [GitHub Issues](#)

Shrub Hub

CHANGING OUTPUTS GUIDE

Changing the Code



The Shrub Hub has several settings that can be changed to adjust its operation.

Modifying these settings involves modifying text in the code file. This should be done with caution, as it is possible to break the code by making changes to it. Always make a backup of the file before making any changes.

Accessing Code to Make Changes

To change the settings on the Shrub Hub, you first must access the code. The code can be found by opening the file explorer on your computer when the Shrub Hub is plugged in, and there should be a drive called CIRCUITPY that appears similar to plugging in a USB flash drive. Clicking on this drive should show a folder called lib, and a file called code.py. The code is stored in the code.py file, which can be opened using notepad or a code editing program. The program Mu was used to develop the code, and it can be downloaded at this link: [Mu Code Editor](#).

After opening code.py in your program of choice, the settings for the Shrub Hub are found in the settings section of the code, which starts here, labeled with SETTINGS and DO NOT CHANGE ANYTHING ABOVE THIS LINE.

```
gc.collect()
## SETTINGS =====
## DO NOT CHANGE ANYTHING ABOVE THIS LINE

# This section contains the settings that can be changed in the Shrub Hub. Each setting has a short comment identifying it, and what it
# does, more detailed instructions on each one and how best to change it can be found in the Shrub Hub User Guide, found here
# https://github.com/makersmakingchange/Shrub-Hub/blob/main/Documentation/Shrub_Hub_User_Guide.pdf
# Do not change any code outside of this marked settings section.

modeChangeTime = 4 #Number of seconds a switch must be held down to change the mode

numModes = 5 #Number of modes. If adding a new one, make sure to add a new colour in setModeColour
```

And ends here, labeled with END OF SETTINGS and DO NOT CHANGE ANYTHING BELOW THIS LINE.

```
| | | | [[3, [ConsumerControlCode.PLAY_PAUSE]], [3, [ConsumerControlCode.VOLUME_INCREMENT]], [3, [ConsumerControlCode.VOLUME_DECREME
# END OF SETTINGS =====
## DO NOT CHANGE ANYTHING BELOW THIS LINE
```

In this section, there are four user changeable settings. These settings are modeChangeTime, numModes, colourMatrix, and keystrokeMatrix.

Changing Mode Change Time

modeChangeTime is the simplest setting to change. This setting controls the number of seconds that a switch needs to be held down before it changes the mode. The default for this is four seconds and changing it will change the amount of time the switch needs to be held down.

```
## SETTINGS =====
## DO NOT CHANGE ANYTHING ABOVE THIS LINE

# This section contains the settings that can be changed in the Shrub Hub. Each set
# does, more detailed instructions on each one and how best to change it can be fou
# https://github.com/makersmakingchange/Shrub-Hub/blob/main/Documentation/Shrub_Hub
# Do not change any code outside of this marked settings section.

modeChangeTime = 4 #Number of seconds a switch must be held down to change the mode

numModes = 5 #Number of modes. If adding a new one, make sure to add a new colour in
```

Changing the Number of Modes

numModes is the number of modes that the Shrub Hub has. This is currently set to five. To change this, you also need to change the number of rows in both of the next two settings. If it is less than the number of rows, you will never be able to access some of the modes, and if it is greater than the number of rows the Shrub Hub will crash and stop working when you try and change the mode. For example, if this is set to three without changing anything else, it will be impossible to access modes four and five. If it is set to six, the Shrub Hub will crash when trying to change out of mode five.

```
## SETTINGS =====
## DO NOT CHANGE ANYTHING ABOVE THIS LINE

# This section contains the settings that can be changed in the Shrub Hub. Each setting has a short c
# does, more detailed instructions on each one and how best to change it can be found in the Shrub Hu
# https://github.com/makersmakingchange/Shrub-Hub/blob/main/Documentation/Shrub_Hub_User_Guide.pdf
# Do not change any code outside of this marked settings section.

modeChangeTime = 4 #Number of seconds a switch must be held down to change the mode

numModes = 5 #Number of modes. If adding a new one, make sure to add a new colour in the colourMatrix

# These are the colours for each mode. Each colour is a list of three values, ordered red, green, blue
# Online RGB calculators can be used to help you determine the correct values for each for a given col
# Each of the RGB values can be anywhere from 0 to 255.
```

colourMatrix is where the Shrub Hub stores information on what colour the NeoPixel should be in each mode. The colour matrix is organized in a list and can be read as colourMatrix = [mode1colour, mode2colour, mode3colour, mode4colour, mode5colour] with new lines added for legibility. Each individual mode colour is made up of three numbers that contain amount of red, green, and blue (RGB) in each colour, and is formatted in square brackets as [Red, Green, Blue]. These numbers can be anywhere from 0 to 255, and online RGB calculators can be used to determine the correct RGB numbers for any given colour.

```
colourMatrix = [[0, 128, 128], # Mode 1 = Teal
                [50, 0, 128],  # Mode 2 = Purple
                [50, 60, 0],    # Mode 3 = Yellow
                [0, 60, 0],     # Mode 4 = Green
                [120, 60, 0]]   # Mode 5 = Orange
```

Example Colour Change

As an example, if you wanted to change the colour of mode four to be magenta instead of green, these are the steps that you would follow to do so.

The first step is to find the section that you want to change. We want to change the colour of mode four, so we look at the fourth row, helpfully labelled with the comment Mode 4 = Green. This row contains the list [0, 60, 0], which is the RGB colour code of the mode. The R stands for red, and the first entry is zero, which means that there is no red. The G stands for green, and the second entry is 60, which means the green is set to 60 out of a maximum of 255. Finally, the B stands for blue, and the third entry is set to zero which means that there is no blue. This leaves us with only a green light set to roughly one third power as to not be too bright.

CHANGING OUTPUTS GUIDE

The next step is to determine the colour code for the new colour we want. Since we are going for a magenta colour, we search online to find the colour code. The RGB of magenta is found to be 255, 0, 255, which is equal parts red and blue. Since this will be quite bright, we will turn it down to 128, 0, 128. The balance between the colours is not changing, we are just turning down the brightness.

Finally, we go into the colourMatrix to replace the old RGB code for mode 4 with the new code. We replace the [0, 60, 0] with [128, 0, 128], and update the comment to show the new mode colour. The final results should look like this.

```
colourMatrix = [[0, 128, 128], # Mode 1 = Teal
                [50, 0, 128],  # Mode 2 = Purple
                [50, 60, 0],    # Mode 3 = Yellow
                [128, 0, 128],  # Mode 4 = Magenta
                [120, 60, 0]]   # Mode 5 = Orange
```

If the user wants different outputs than the defaults, it is possible to change them. The outputs for the different mode are stored in the `keystrokeMatrix` variable. The keystroke matrix is organized in a list and can be read as `keystrokeMatrix = [mode1keystrokes, mode2keystrokes, mode3keystrokes, mode4keystrokes, mode5keystrokes]`.

Each mode keystrokes is also made up of a list that is structured like `[switch1keystroke, switch2keystroke, switch3keystroke, switch4keystroke, switch5keystroke]`. This leads to a matrix that looks like the following,

`keystrokeMatrix =`

```
[[switch1keystroke, switch2keystroke, switch3keystroke, switch4keystroke, switch5keystroke],
[switch1keystroke, switch2keystroke, switch3keystroke, switch4keystroke, switch5keystroke],
[switch1keystroke, switch2keystroke, switch3keystroke, switch4keystroke, switch5keystroke],
[switch1keystroke, switch2keystroke, switch3keystroke, switch4keystroke, switch5keystroke],
[switch1keystroke, switch2keystroke, switch3keystroke, switch4keystroke, switch5keystroke]]
```

Finally, each individual keystroke is made of a set of two lists that look like this `[identifier, [keystroke1, keystroke2, etc.]]`. The identifier is either 1, 2, or 3, and it tells the program if it is a keystroke(1), a mouse click(2), or a media control command(3). The list of keystrokes will commonly be a single keystroke, but there is support for multiple keystrokes if you want to have the device send a CTRL + C or CTRL + SHIFT + T for example. Example valid entries can look like `[1, [Keycode.TAB]]`, `[1, [Keycode.GUI, Keycode.H]]`, `[1, [Keycode.CONTROL, Keycode.SHIFT, Keycode.T]]`, `[2, [Mouse.LEFT_BUTTON]]`, `[3, [ConsumerControlCode.PLAY_PAUSE]]`

The full list of valid keycodes can be found at the following link: [Keycode List](#).

Changing a Keystroke

As an example of changing a keystroke, we are going to look at changing Output 3 on Mode 2 from F3 to a 3.

The first step is to find the output that you want to change. Since we are looking to change something in Mode 2, find the second row of the `keystrokeMatrix`. In the second row, find the third entry. This may sound complicated, but it is made easier by the fact that the entries are separated by commas, and often contain the name of the keystroke in capital letters. In this case, we are looking for the entry `[1, [Keystroke.F3]]`. Since we are just looking to change a keystroke, we can leave the 1 alone and focus on the `Keystroke.F3`.

```
keystrokeMatrix = [[1, [Keystroke.ENTER]], [1, [Keystroke.SPACE]], [1, [Keystroke.TAB]], [1, [Keystroke.BACKSPACE]], [1, [Keystroke.ESCAPE]],
                  [1, [Keystroke.F1]], [1, [Keystroke.F2]], [1, [Keystroke.F3]], [1, [Keystroke.F4]], [1, [Keystroke.F5]], #purple, Mode 2
                  [2, [Mouse.LEFT_BUTTON]], [2, [Mouse.RIGHT_BUTTON]], [2, [Mouse.MIDDLE_BUTTON]], [2, [Mouse.LEFT_BUTTON]], [2,
                  [1, [Keystroke.F13]], [1, [Keystroke.F14]], [1, [Keystroke.F15]], [1, [Keystroke.F16]], [1, [Keystroke.F17]], #green, Mo
                  [3, [ConsumerControlCode.PLAY_PAUSE]], [3, [ConsumerControlCode.VOLUME_INCREMENT]], [3, [ConsumerControlCode.VOL
```

Now that we have found the output that we want to change, the next step is to find what we want to change it to. Keycodes for every output can be found at the following link: [Keystroke List](#). Since we want to change it to a 3, we go to the keycode section of the linked site and find that the code for a 3 is `Keystroke.THREE`.

Now that we have our new keycode and we know where it goes, we simply replace the `Keystroke.F3` with `Keystroke.THREE` so that entry looks like `[1, [Keystroke.THREE]]`. Save the code and now pressing the switch connected to Switch Input Port 3 while in mode 2 should send the character 3 instead of F3.

Changing an Output Type

A more complicated change would be to change a keystroke to a mouse click or a media control command. For this, we are going to look at changing Output 3 on Mode 2 from F3 to the media control command mute.

Following the steps from the previous section, find the section that you want to change. In our case, it is the entry `[1, [Keystroke.F3]]`. Since we are changing it from keystrokes to media controls, we have to change more things than the previous section. The first thing to change is the mode indicator, which is the 1 at the beginning of the entry. The 1 tells the program that it is a keycode, we want to change that to a 3 since we are changing to a media command. If we were changing it to a mouse command we would use a 2.

The next step is to change the code prefix. On the current entry it is `Keystroke`, but since we are changing it to a media control command we want to change it to `ConsumerControlCode`. If we were changing it to a mouse command we would change it to `Mouse`.

CHANGING OUTPUTS GUIDE

Finally, we need to find what command we want to change it to. The different media control commands can be found at the following link: [Command List](#). Scrolling down to the ConsumerControlCode section on the page, we find that the code for mute is MUTE.

Now that we know the three things that we need to change, we change the [1, [Keycode.F3]] to [3, [ConsumerControlCode.MUTE]]. Save the code, and now pressing the switch connected to Switch Input Port 3 while in mode 2 should mute or unmute the audio of the device instead of sending the character F3.

Output Link

The keycodes, mouse clicks, and consumer control codes can be found here or at the following QR code: <https://docs.circuitpython.org/projects/hid/en/latest/api.html>

